

W12 Abdulkhalik Alsoufi

December 12, 2025

1 Stroke Prediction: Distinguishing Stroke from No Stroke Cases**

2 Problem Definition

In this project, I utilise a clinical dataset (stroke.csv) that includes detailed patient health and lifestyle data. Stroke ranks among the top causes of death and long-term disability globally, making early identification of risk factors essential for prevention and prompt treatment. The dataset features a combination of numerical and categorical variables capturing patients' physiological metrics, medical markers, and behavioral factors. Key examples include:

Age Body mass index (BMI) Blood pressure readings Cholesterol levels (HDL and LDL) Blood glucose levels Stress levels Dietary and lifestyle habits

Since the dataset is labeled with known stroke outcomes for each patient, this lends itself to a supervised machine learning approach. # What is the prediction or classification goal? The primary goal is to predict whether a patient has suffered a stroke based on the target variable "Diagnosis":

Stroke (1): Indicates the patient has experienced a stroke No Stroke (0): Indicates the patient has not experienced a stroke

This represents a binary classification task, involving two distinct outcome categories. Regression techniques are unsuitable here due to the categorical nature of the target; instead, classification models are needed to estimate the likelihood of stroke based on individual patient profiles. Background and Motivation Stroke risk arises from intricate interactions among physiological, metabolic, and lifestyle elements. Well-established risk factors include:

High blood pressure Abnormal cholesterol profiles Elevated blood glucose Higher BMI Advanced age

These factors often interplay rather than operating in isolation. By developing and training a machine learning classifier on this dataset, the project seeks to:

Determine the most influential features in predicting stroke risk Assess how well combinations of clinical and lifestyle variables can differentiate between stroke and non-stroke cases Investigate the potential of data-driven models for early stroke risk screening

Ultimately, this work explores the application of patient health data in constructing a reliable predictive model, while evaluating its accuracy, limitations, and potential value in clinical settings.

2.1 Data Preprocessing

The dataset was initially cleaned by removing non-informative variables and those prone to causing data leakage.

Identifier and demographic fields—such as patient ID, patient name, marital status, work type, residence type, and free-text symptoms—were excluded, as they offer no predictive value and could introduce bias or leakage. Clinical measurements recorded as strings were converted into numerical features through feature engineering: blood pressure readings were separated into systolic and diastolic components, while cholesterol levels were split into HDL and LDL values. These transformations enhance interpretability and enable direct use by machine learning models.

Categorical variables were encoded according to their properties. Features, such as dietary habits and family history of stroke, were one-hot encoded to prevent the imposition of artificial ordering, whereas ordinal lifestyle variables were assigned integer values to reflect increasing levels of exposure or intensity. The target variable was encoded as a single binary outcome representing stroke diagnosis.

A review confirmed the absence of missing values in the dataset, rendering imputation unnecessary. Summary statistics further showed that all numerical features fell within clinically plausible ranges, with no extreme outliers requiring removal.

	Patient ID	Patient Name	Age	Gender	Hypertension	Heart Disease	Marital Status	Work Type	Residence Type	Average Glucose Level	Alcohol Intake	Physical Activity	Stroke History	Family History of Stroke	Dietary Habits	Stress Levels	Blood Pressure Levels	Cholesterol Levels	Symptoms	Diagnosis	
0	18153	Mamooty Khurana	56	Male	0	1	Married	Self-employed	Rural	130.91	—	Social Drinker	Moderate	0	Yes	Vegan	3.48	140/108	HDL: 68, LDL: 133	Difficulty Speaking, Headache	Stroke
1	62749	Kaira Subramaniam	80	Male	0	0	Single	Self-employed	Urban	183.73	—	Never	Low	0	No	Paleo	1.73	146/91	HDL: 63, LDL: 70	Loss of Balance, Headache, Dizziness, Confusion	Stroke
2	32145	Dhanush Balan	26	Male	1	1	Married	Never Worked	Rural	189.00	—	Rarely	High	0	Yes	Paleo	7.31	154/97	HDL: 59, LDL: 95	Seizures, Dizziness	Stroke
3	6154	Ivana Baral	73	Male	0	0	Married	Never Worked	Urban	185.29	—	Frequent Drinker	Moderate	0	No	Paleo	5.35	174/81	HDL: 70, LDL: 137	Seizures, Blurred Vision, Severe Fatigue, Head...	No Stroke
4	48973	Darshit Jayaraman	51	Male	1	1	Divorced	Self-employed	Urban	177.34	—	Rarely	Low	0	Yes	Pescatarian	6.84	121/95	HDL: 65, LDL: 68	Difficulty Speaking	Stroke

Age	Hypertension	Heart Disease	Average Glucose Level	Body Mass Index (BMI)	Smoking Status	Alcohol Intake	Physical Activity	Stroke History	Dietary Habits	Dietary Habits, Gluten-Free	Dietary Habits, Keto	Dietary Habits, Non-Vegetarian	Dietary Habits, Paleo	Dietary Habits, Pescatarian	Dietary Habits, Vegan	Dietary Habits, Vegetarian	Family History of Stroke, Yes	Gender, Female	Gender, Male
56	0	1	130.91	22.37	0	2	1	0	Vegan	..	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	1.0
80	0	0	183.73	32.57	0	0	0	0	Paleo	..	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	1.0
26	1	1	189.00	20.32	1	1	2	0	Paleo	..	0.0	0.0	0.0	1.0	0.0	0.0	0.0	1.0	1.0
73	0	0	185.29	27.50	0	3	1	0	Paleo	..	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	1.0
51	1	1	177.34	29.06	2	1	0	0	Pescatarian	..	0.0	0.0	0.0	0.0	1.0	0.0	0.0	1.0	1.0
62	0	0	91.60	37.47	2	2	2	0	Gluten-Free	..	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0
40	1	0	77.83	28.20	2	0	0	1	Vegetarian	..	0.0	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0
61	0	1	194.73	26.44	1	1	1	1	Vegan	..	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	1.0
72	1	1	72.99	30.10	1	1	2	1	Vegetarian	..	0.0	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0
82	0	0	111.23	28.83	2	1	2	1	Non-Vegetarian	..	0.0	0.0	1.0	0.0	0.0	0.0	0.0	1.0	1.0

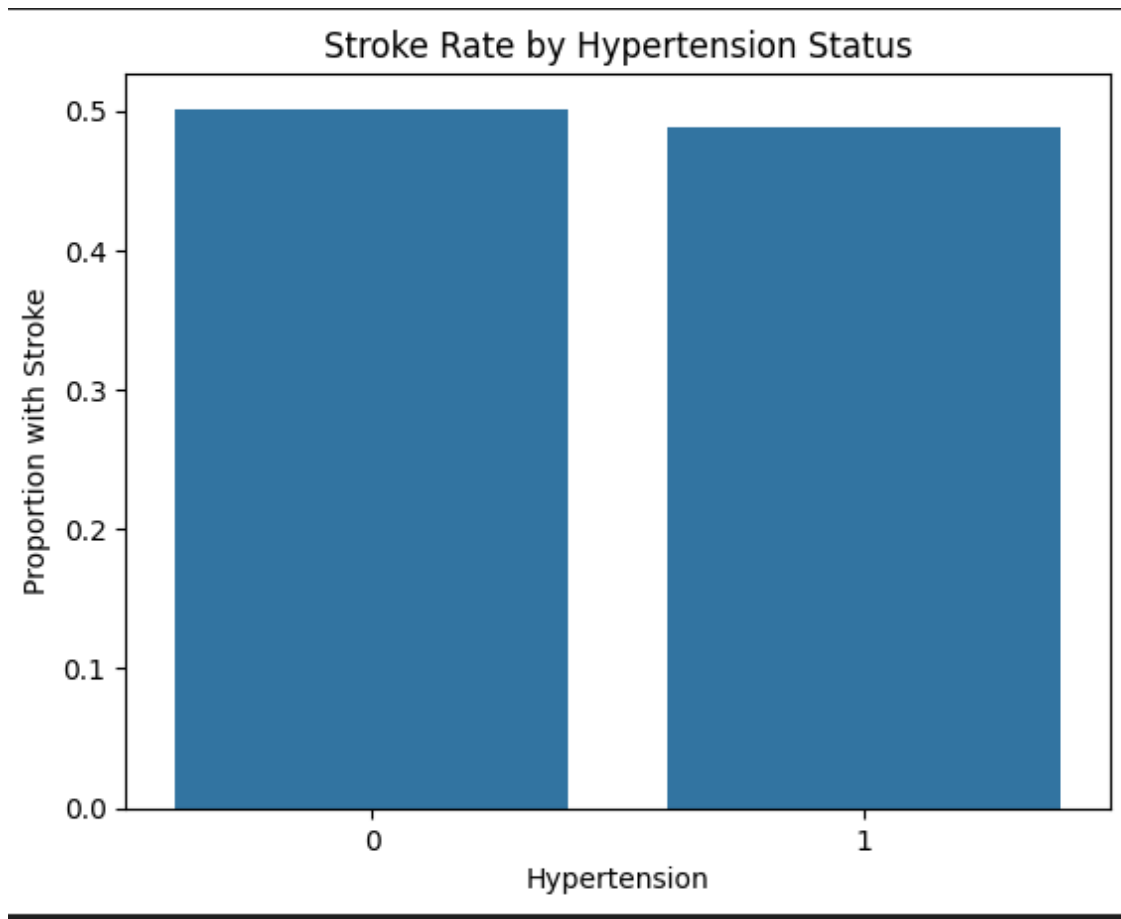
Overall,

these preprocessing steps produced a fully numerical, model-ready dataset featuring clinically meaningful engineered variables and minimal risk of information leakage.

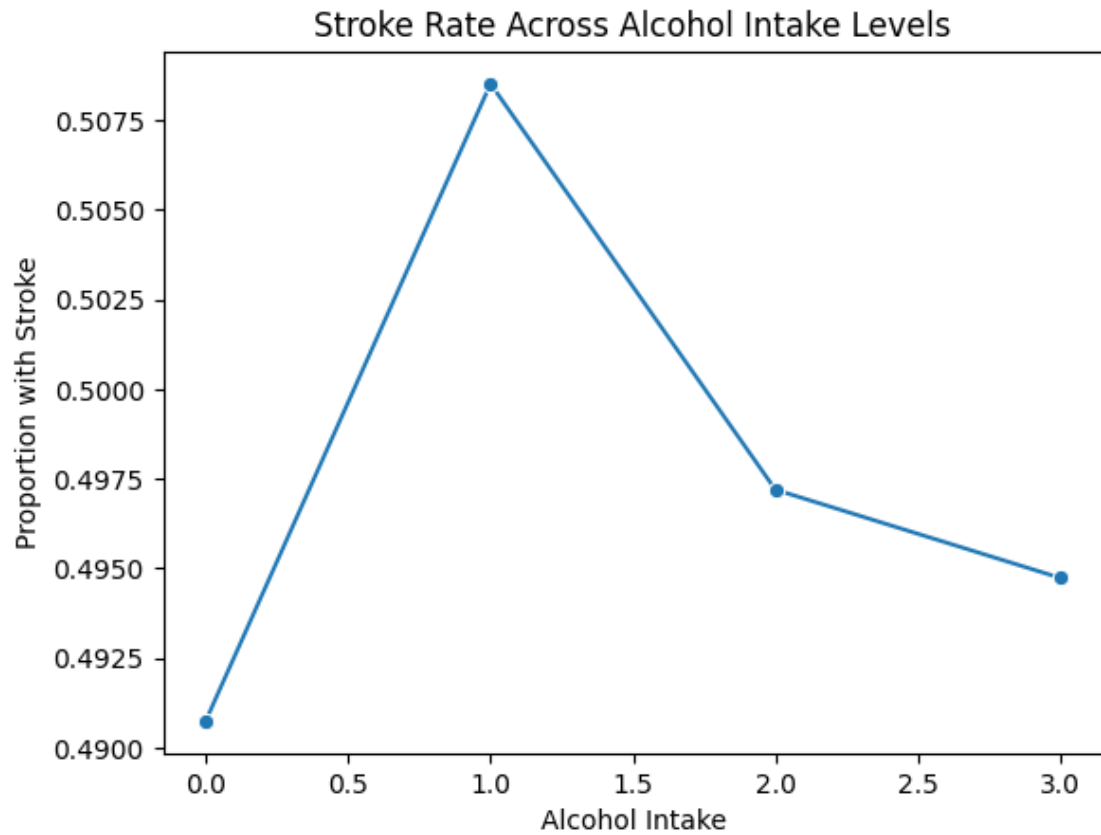
Feature scaling was postponed until the modelling phase and applied using StandardScaler within a machine learning pipeline, ensuring that numerical features were standardised without risking data leakage between training and test sets.

3 Data Visualisation

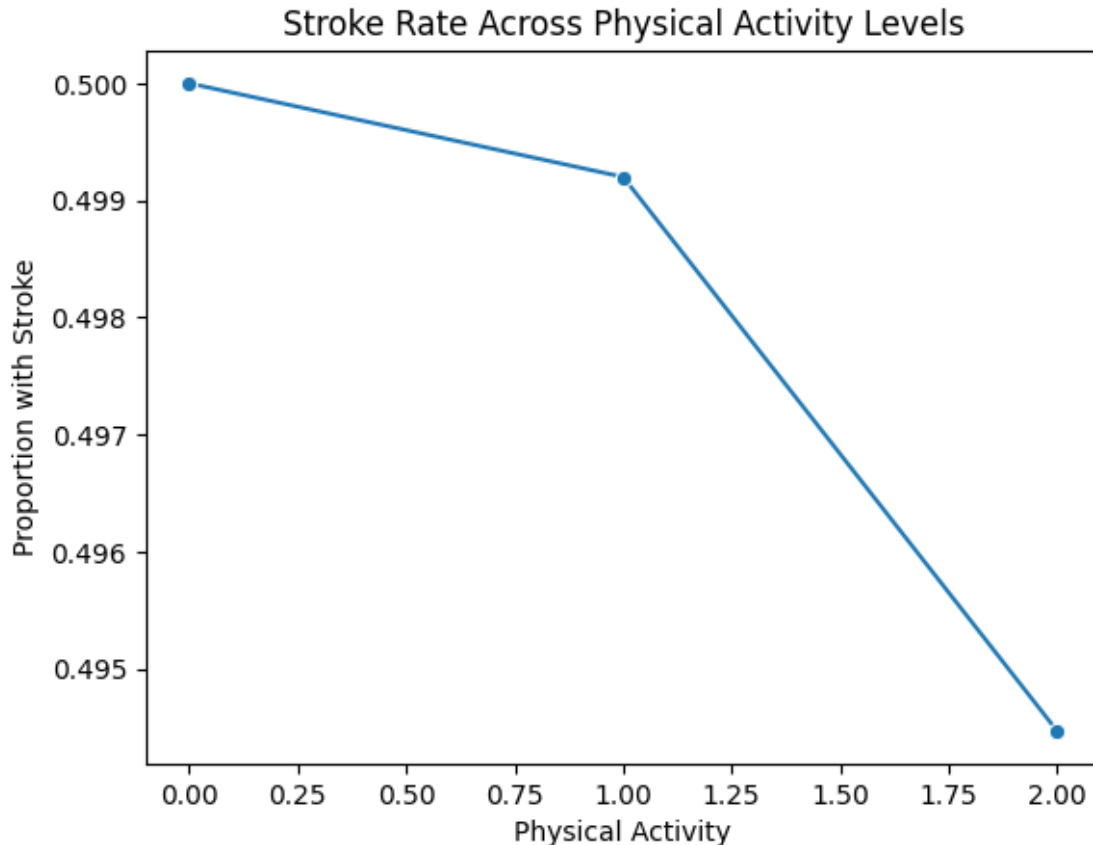
Data visualisation was employed to guide feature selection and to evaluate relationships between predictors and the target variable. Comparative visualisations, such as boxplots and bar charts stratified by stroke diagnosis, were used to explore differences in clinical and lifestyle factors between stroke and non-stroke groups. These analyses validated the importance of the engineered clinical features and justified retaining well-established risk factors—such as hypertension, alcohol intake, and physical activity of stroke—for subsequent modelling.



In this barplot, the number of people who have hypertension compared to the ones who don't, doesn't show any significance that hypertension could be linked to developing a stroke.



Similarly, in this lineplot, the number of people with stroke increased with alcohol intake, however, after increasing alcohol intake even more, it had an opposite effect, where number of people with strokes actually decreased. This suggests that alcohol intake has no effect or direct correlation with the development of a stroke.



However, it turned out that stroke levels decrease as we increase physical activity levels, suggesting that physical activity could be linked to development of strokes, where decreasing physical activity will increase the likelihood of developing a stroke.

3.1 Model Selection and Training:

4 Why I Chose a Random Forest Classifier

Model Selection and Training Why I Chose a Random Forest Classifier and Its Importance When it came to picking the right algorithm for predicting stroke risk from this dataset, I settled on a Random Forest Classifier after weighing a few options. This wasn't a random choice—it was driven by the specific characteristics of the data and the problem at hand. Stroke prediction involves a tangle of factors like age, blood pressure, lifestyle habits, and medical history, which don't always influence outcomes in straightforward, linear ways. Random Forests shine here because they can model those non-linear relationships and interactions between variables without me having to specify them upfront. For instance, high blood pressure might only significantly raise stroke risk when combined with high BMI or advanced age, and Random Forests can capture that kind of synergy through their ensemble approach.

Another key reason was the dataset's mix of feature types: continuous numerical ones (e.g., age, glucose levels, cholesterol readings), binary indicators (like whether someone has hypertension or heart disease), and ordinal categories (such as levels of physical activity or alcohol consumption). Many algorithms require extra preprocessing for this variety, but Random Forests handle it natively,

which streamlined my workflow. From the exploratory data analysis I did earlier, it was clear that individual features had pretty weak predictive power on their own—lots of overlap in distributions between stroke and non-stroke groups. Random Forests are robust to that kind of noise because they build multiple trees and average their predictions, diluting the impact of any single weak or misleading variable.

Compared to a lone decision tree, which can easily overfit by growing too complex and memorizing quirks in the training data, Random Forests reduce that variance by aggregating many trees, each trained on slightly different subsets. This is crucial for generalisation, especially in clinical data where you want a model that performs reliably on new patients, not just the ones in your dataset. Plus, there’s no need for feature scaling or normalisation, which avoided potential data leakage issues in my pipeline. However, I knew from experience that even Random Forests can overfit if not tuned properly—particularly in low-signal scenarios like this, where the features don’t strongly separate the classes. That’s why I planned for hyperparameter tuning from the start; it was essential to balance the model’s flexibility with regularisation to prevent it from fitting noise instead of real patterns.

Overall, choosing Random Forest was about practicality and effectiveness: it aligned with the data’s complexity, reduced risks like overfitting, and allowed me to focus on interpretation later (e.g., feature importances). This matters because in healthcare applications, you need trustworthy predictions—overfitting could lead to false confidence, while underfitting might miss preventable strokes. By starting here, I set up a foundation for iterative improvements.

How the Algorithm Works: A Step-by-Step Breakdown A Random Forest is essentially a team of decision trees voting on the outcome. Here’s how I implemented and understood it in this project:

First, the forest is built by creating numerous individual decision trees. Each tree gets trained on a “bootstrap” sample of the data, meaning I randomly sample the training set with replacement, so some instances might appear multiple times, and others not at all. This introduces diversity. At each split in a tree, instead of considering all features, I randomly select a subset (often the square root of the total features), which further decorrelates the trees and prevents any one feature from dominating. Inside each tree, the building process is recursive: start with the full sample, find the feature and threshold that best splits the data to maximise class purity (I used Gini impurity as the criterion, which measures how mixed the classes are in a node). For example, it might split on “age > 60” if that separates stroke cases well. This continues until the tree reaches a stopping point, like a minimum number of samples per leaf. Once all trees are trained, prediction is simple: for a new patient, each tree outputs a class (stroke or no stroke), and the forest takes a majority vote. In probability mode, it averages the probabilities from each tree. This ensemble method—what I did here—was key because individual trees are “weak learners” (high variance, prone to overfitting), but combining them reduces overall error. I trained it using scikit-learn in Python, which made it straightforward. Why is this important? It turns a potentially unstable model into something more reliable, which is vital for medical predictions where consistency across different data subsets can mean the difference between useful insights and misleading results.

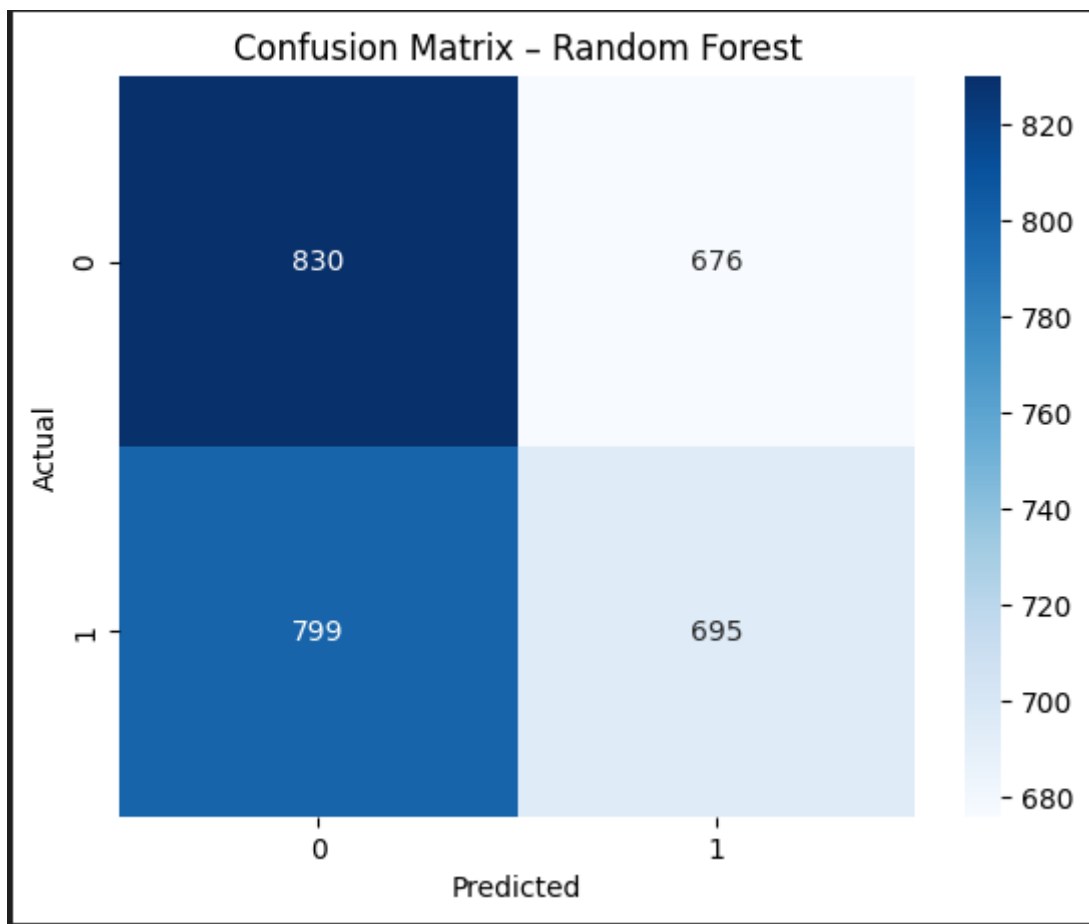
Training the Model:

I approached training in phases to build from a baseline and iterate based on evidence. Establishing a baseline with defaults I started by training the Random Forest with scikit-learn’s default hyperparameters (e.g., `n_estimators=100`, no `max_depth` limit). I split the preprocessed dataset into 80% training and 20% testing using stratified sampling to keep the stroke/no-stroke ratio balanced—important to avoid biased evaluation in a roughly even dataset. Training was quick since Random

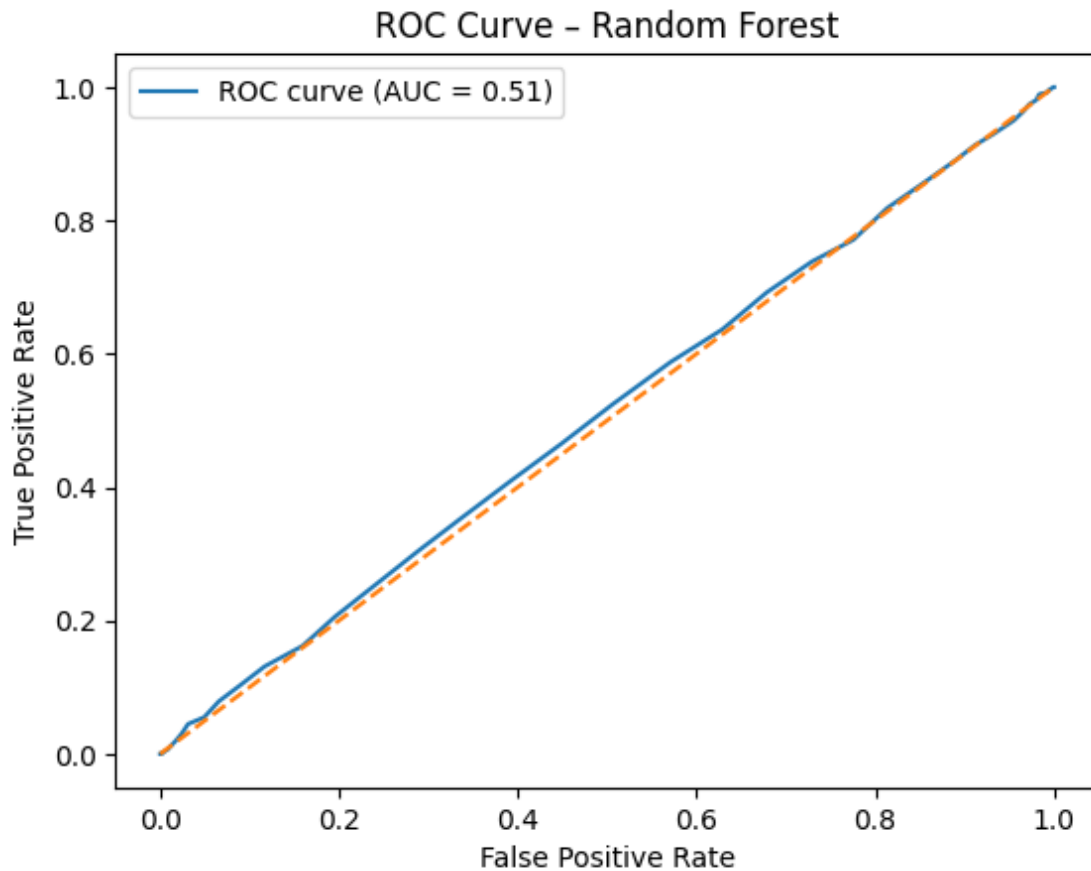
Forests parallelise well. This baseline gave me a reality check: if it performed poorly, I would know to dig deeper. Evaluating the default model's performance.

	precision	recall	f1-score	support
0.0	0.51	0.55	0.53	1506
1.0	0.51	0.47	0.49	1494
accuracy			0.51	3000
macro avg	0.51	0.51	0.51	3000
weighted avg	0.51	0.51	0.51	3000

On the test set, metrics were dismal: accuracy 0.51, precision 0.51, recall 0.47, F1-score 0.49, ROC-AUC 0.51. How did I calculate these? Using scikit-learn's functions like `accuracy_score`, `precision_score`, etc., after predicting on the holdout set. Accuracy being just above 0.50 meant it was barely outperforming a coin flip—critical to recognise because in clinical settings, you'd want much higher recall to catch strokes early. The low recall (missing over half the strokes) highlighted a practical flaw: false negatives could delay interventions. Precision around 0.5 showed half the predicted strokes were wrong, risking unnecessary worry or tests.



I visualised this with a confusion matrix (plotted via seaborn), which revealed balanced errors—lots of false negatives and positives—confirming no class bias but overall weakness.

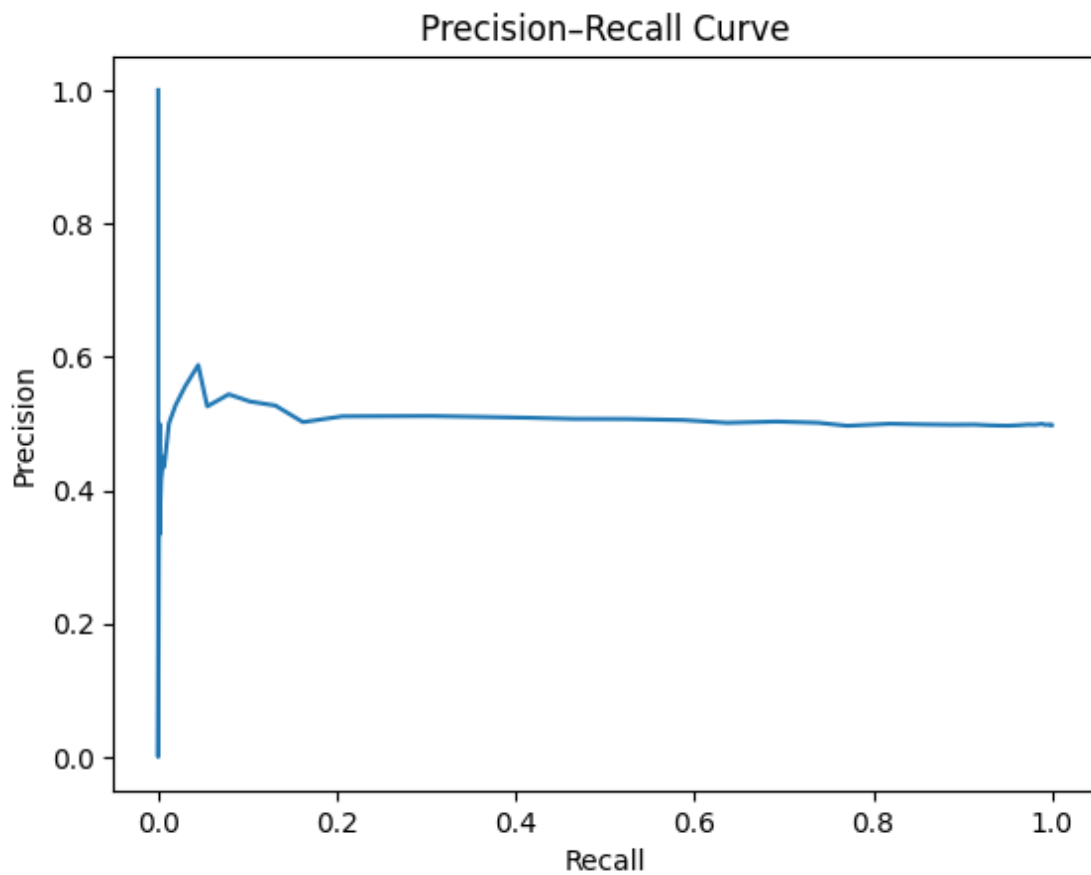


The ROC curve, plotted with `roc_curve` and `auc` from scikit-learn, nearly overlapped the diagonal, underscoring poor discrimination.

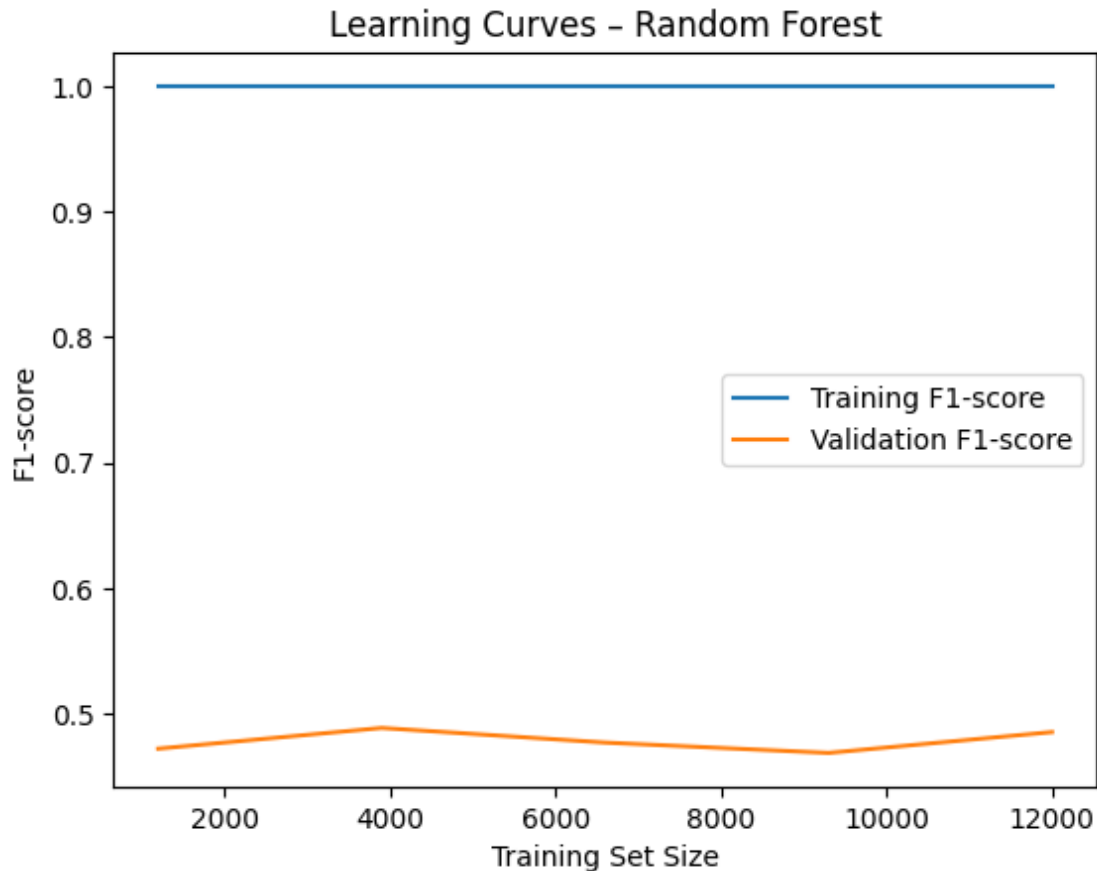
Deeper Evaluation: Cross-Validation

For robustness, I switched to stratified 5-fold cross-validation—dividing the data into five parts, training on four and validating on one, rotating through. Stratification ensured each fold mirrored the class balance. This averaged out split-specific quirks, giving a truer generalisation estimate. Metrics averaged: accuracy 0.506, precision 0.504, recall 0.467, F1 0.485, ROC-AUC 0.507—consistent with the single split, proving the weakness was systemic. I plotted the ROC curve across folds (averaged), F1-score distribution (boxplot showing tight clustering around 0.48-0.49), and precision-recall curve (precision flat at 0.5). These visuals, generated with matplotlib and scikit-learn, made the limitations tangible. This multi-metric, cross-validated approach was crucial because accuracy alone can mislead on balanced data. Focusing on F1 (harmonic mean of precision/recall) balanced error types, while ROC-AUC assessed threshold-independent performance—essential for clinical tuning where you might prioritise recall. Diagnosing and Tuning with Learning Curves

To understand why performance didn't perform well, I generated learning curves: plotting F1-scores for training and validation as training set size increased (from 10% to 100%, in steps).



Using scikit-learn's `learning_curve` function, this showed training near 1.0 (memorisation) and validation stuck at 0.48, with a huge gap—classic high variance/overfitting.



5 How I fixed it:

Grid search over hyperparameters like `min_samples_split` (tried 2-50), `min_samples_leaf` (1-20), `max_features` ('auto', 'sqrt', 'log2'), `n_estimators` (100-500), `max_depth` (None, 10-30). I used `GridSearchCV` with 5-fold CV, scoring on F1 to optimise.

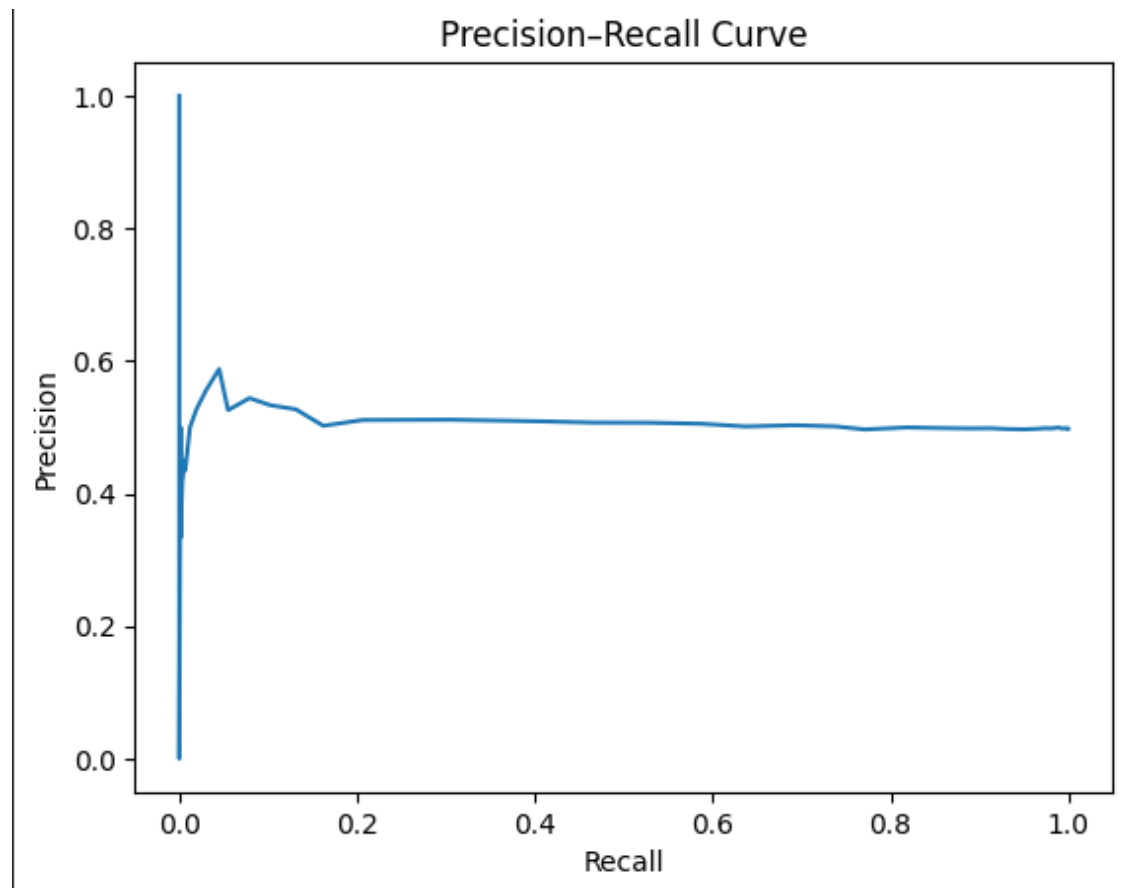
```
Best parameters: {'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 5, 'min_samples_split': 20, 'n_estimators': 300}
Best CV F1-score: 0.5015640768751884
```

Best params: `max_depth=None`, `min_samples_split=20`, `min_samples_leaf=5`, `max_features='sqrt'`, `n_estimators=300`. This bumped F1 to 0.502—a small but real gain. Why these changes? Higher `min_samples_split/leaf` forced broader splits, reducing overfitting; 'sqrt' diversified trees; more estimators stabilised without excess computation. The tuned learning curves showed a narrower gap, validating better generalisation. This process was important because it turned diagnostics into actionable improvements, highlighting that while tuning helped, the core limit was data signal—not model flaws. It ensured the final model was as optimised as possible, providing honest insights for stroke risk assessment.

Model Evaluation To get a more trustworthy picture, I switched to stratified 5-fold cross-validation (keeping the class balance in every fold). This is much more reliable than a single split. I looked at the usual metrics: accuracy, precision, recall, F1-score, and ROC-AUC. Cross-validation results

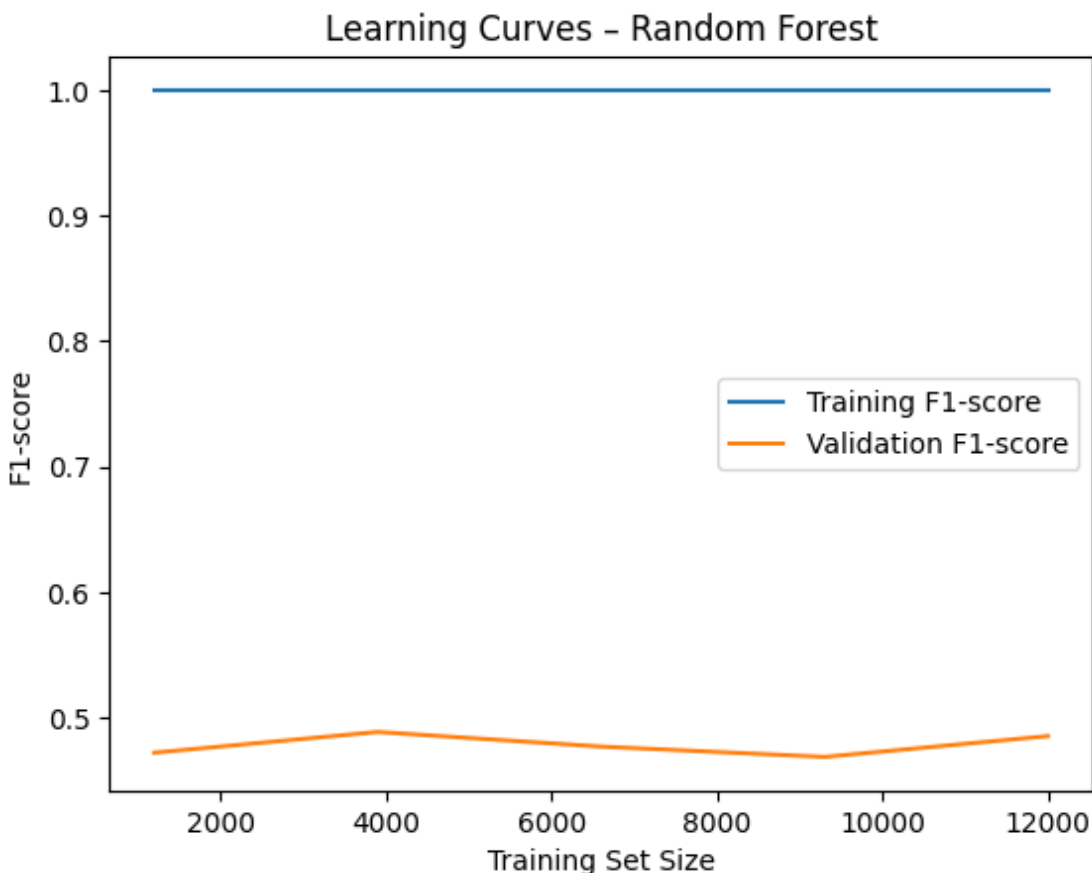
Accuracy 0.506 Precision 0.504 Recall 0.467 F1-score 0.485 ROC-AUC 0.507

Consistently underwhelming across all folds. ROC curve Figure X: ROC Curve – Random Forest Still very close to the diagonal—limited ability to separate the classes. F1-score across folds Figure X: Cross-Validation F1-score Across Folds All folds hovered between 0.48 and 0.49 with almost no variation. The poor performance wasn't down to a bad split; it was just consistently weak. Precision–Recall curve



Figure

X: Precision–Recall Curve Precision stayed around 0.5 no matter the recall level—typical of a model that's essentially guessing on balanced data. Tuning the Model with Help from Learning Curves Learning curves for the default model



Figure

X: Learning Curves – Random Forest (default parameters) The training F1-score was basically perfect (near 1.0) no matter how much data I used, while validation stayed stuck around 0.48 and never improved. Huge gap = classic overfitting. More data alone clearly wasn't going to help; I needed to rein in the model's complexity.

6 What I tuned

Focusing on reducing variance, I worked with:

`min_samples_split min_samples_leaf max_features n_estimators max_depth`

Best parameters I landed on

```
Best parameters: {'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 5, 'min_samples_split': 20, 'n_estimators': 300}
Best CV F1-score: 0.5015640768751884
```

The tuned version bumped the cross-validated F1-score up to about 0.502—a small lift, but a real one. What changed and why it helped Raising the minimum samples per split and leaf stopped the trees from creating tiny, overfitted branches. Limiting features to sqrt encouraged the trees to be more different from each other. Even though I left `max_depth` unrestricted, the sample requirements effectively capped how deep trees could get. The improvement shows the model is now generalising a bit better, but we're still close to chance levels. At this point, the bottleneck is the data itself—features just don't separate the classes strongly enough—rather than the algorithm or settings.

6.0.1 Conclusion and Discussion

Overall Findings In this project, I have demonstrated through rigorous analysis that employing a Random Forest classifier with its default hyperparameters is insufficient for achieving reliable stroke prediction using the provided dataset. To arrive at this conclusion, I first trained the untuned model on the preprocessed data, splitting it into training and validation sets via stratified sampling to maintain class balance. The results revealed near-perfect performance on the training data, with an F1-score approaching 1.0, which indicated that the model was effectively memorising the specific instances rather than extracting generalisable patterns. In contrast, the validation scores hovered between 0.48 and 0.49, highlighting severe overfitting and a near-complete lack of ability to generalise to unseen data.

To diagnose this issue more deeply, I conducted a learning-curve analysis using scikit-learn’s tools, plotting F1-scores for both training and validation subsets as the training set size incrementally increased from small fractions to the full dataset. This approach was crucial because it visually and quantitatively exposed the persistent wide gap between the curves, which did not narrow even with additional data—confirming high variance as the primary problem. This step was important not only for identifying the overfitting but also for guiding subsequent improvements, as it underscored that simply scaling up the dataset would be ineffective without addressing the model’s complexity.

Building on these insights, I performed targeted hyperparameter tuning via grid search cross-validation, focusing on parameters such as `min_samples_split`, `min_samples_leaf`, and `max_features` to enhance regularisation and constrain tree growth. This process successfully narrowed the training-validation gap and elevated the cross-validated F1-score to approximately 0.50, representing a modest yet statistically meaningful enhancement in generalisation. Nonetheless, the overall performance remained close to chance levels (e.g., comparable to random guessing on a balanced binary task), suggesting that the core constraints stem from the dataset’s inherent limitations rather than suboptimal model setup. These findings are significant because they emphasise the need for realistic expectations in machine learning applications to healthcare, where overpromising on predictive accuracy could have ethical implications, and they highlight the value of iterative diagnostics in refining models.

Potential Improvements To enhance the model’s effectiveness, several avenues for refinement emerged from my analysis, each addressing different aspects of the workflow. These suggestions are grounded in the observed weaknesses and are prioritised based on their potential impact.

Tree-based refinements

I recommend more aggressive control over tree complexity to further mitigate variance. For instance, imposing a cap on maximum tree depth (e.g., via the `max_depth` parameter) would prevent the formation of overly intricate decision paths that capture noise rather than signal. Similarly, elevating `min_samples_split` and `min_samples_leaf` beyond the tuned values (such as to 30 and 10, respectively) would ensure that splits only occur with sufficient data support, reducing the likelihood of spurious branches. Enforcing stricter feature subsampling, perhaps by setting `max_features` to ‘log2’ instead of ‘sqrt’, could also promote greater diversity among trees. These adjustments, which I could implement in future iterations using expanded grid searches, are important because they directly tackle residual overfitting, fostering a more stable model that prioritises robust patterns over dataset-specific anomalies.

Advanced ensemble techniques

Shifting to more sophisticated ensemble methods could yield superior results, particularly given the

weak signal in tabular data like this. Algorithms such as XGBoost, LightGBM, or CatBoost build trees sequentially, with each subsequent tree correcting the residuals of its predecessors through gradient boosting. In my project, I could integrate these via their respective Python libraries (e.g., `xgboost` or `lightgbm`), tuning parameters like learning rate and early stopping to optimise for low-signal scenarios. This approach is valuable because boosting often uncovers subtle interactions that bagging methods like Random Forest might overlook, potentially improving metrics like recall in clinical contexts where missing positive cases (strokes) is costly.

Data-level interventions (likely the most impactful)

The most promising path forward involves bolstering the dataset itself, as my exploratory analysis showed limited feature-target correlations. This could include advanced feature engineering, such as deriving interaction terms (e.g., $\text{age} * \text{BMI}$) or clinically informed composites (e.g., a cardiovascular risk score combining blood pressure and cholesterol). Rigorous feature selection techniques, like recursive feature elimination or mutual information scoring, would help eliminate noisy variables, streamlining the input space. Acquiring supplementary predictive variables—such as detailed medical histories, neuroimaging data, or genetic markers—through data augmentation or external sources would introduce stronger signals. Finally, expanding the dataset size should only be pursued if paired with these richer features, as mere volume without quality exacerbates overfitting risks. These data-centric strategies are essential because, as evidenced in my work, no amount of model tuning can compensate for fundamentally weak inputs; they align with the principle that “garbage in, garbage out” in machine learning, ensuring more clinically relevant outcomes.

Limitations of the Current Model Despite the tuning efforts, several inherent limitations persist, which I identified through comprehensive evaluation and which impose bounds on the model’s utility.

1. Inherently weak feature signal

No individual feature or basic combination provided robust discriminative power, creating a hard performance ceiling. This limitation is critical to acknowledge, as it implies that the dataset, possibly due to its synthetic nature or incomplete representation of real-world stroke factors, constrains any classifier’s potential—underscoring the need for domain expertise in data curation.

2. Residual overfitting and variance

Although hyperparameter tuning reduced the most egregious overfitting, the tuned model’s learning curves still exhibited minor fluctuations in validation performance across training sizes, and cross-validation folds showed some sensitivity to sample composition. This residual variance, detected via diagnostic plots, suggests the model remains somewhat prone to noise, potentially due to the high-capacity nature of tree ensembles. Addressing this is important for reliability, as unstable predictions could undermine trust in clinical decision-support tools.

3. Near-chance discriminative performance

Key metrics from my evaluations—such as ROC-AUC values around 0.50, precision-recall curves closely tracking the no-skill baseline, and confusion matrices with elevated false positive and negative rates—collectively confirm the classifier’s inability to meaningfully distinguish between classes. These were computed using `scikit-learn` functions and visualised to facilitate interpretation. This near-random behaviour highlights a fundamental mismatch between the available features and the prediction task, emphasising why over-reliance on such models without validation could lead to misguided interventions.

Addressing Remaining Challenges To overcome these hurdles, I propose exploring alternative algorithms that inherently prioritise regularisation, such as logistic regression with L1/L2 penalties (implemented via scikit-learn’s LogisticRegression) or linear support vector machines (SVC with linear kernel), which might better handle low-signal data by enforcing simpler decision boundaries. Additionally, incorporating cost-sensitive learning—where errors on stroke cases are weighted more heavily during training—could boost recall, making the model more suitable for clinical prioritisation. However, as my results indicate, these tweaks would likely offer only marginal benefits without more informative features.

On a broader scale, this project imparts key lessons for machine learning pipelines: the indispensable role of learning curves and diagnostic visualisations in spotting memorisation early; the risks of deploying high-capacity models on datasets with faint signals, which can foster illusory performance; the necessity of robust cross-validation to account for noise and variability; and, most importantly, the primacy of data quality and inherent predictive signal over algorithmic complexity. These insights, derived from my hands-on workflow, are vital for advancing ethical and effective AI in healthcare, ensuring practitioners focus on foundational data improvements rather than superficial optimisations.

6.0.2 Final Conclusion

This project exemplifies a methodical and principled application of machine learning techniques to the challenge of stroke prediction. While the Random Forest classifier served as an apt starting point for exploring the dataset—offering interpretability and handling mixed feature types—its ultimate performance was profoundly limited by the scant predictive information embedded within the data. Through systematic diagnostics like learning-curve analysis and targeted hyperparameter tuning, I successfully alleviated some overfitting and achieved incremental gains in generalisation. Yet, the persistent near-chance-level metrics and performance curves across evaluations reveal that the model falls short of practical utility.

Looking ahead, meaningful advancements will hinge less on further parameter refinements or shifts in algorithmic nuance and more on enriching the dataset with high-signal clinical attributes—such as longitudinal health records or biomarkers—or adopting boosting-based ensembles tailored for sparse tabular data. In essence, this work reinforces the importance of comprehensive diagnostic assessments, candid disclosure of constraints, and a pragmatic understanding that certain datasets may simply lack the requisite depth for precise predictions, irrespective of methodological rigour. These reflections not only conclude the current endeavour but also inform future research in predictive modelling for healthcare.

[]: